

UDP

Материал из Википедии — свободной энциклопедии

Текущая версия страницы пока не проверялась опытными участниками и может значительно отличаться от версии, проверенной 19 октября 2018 года; проверки требуют 40 правок.

UDP (англ. *User Datagram Protocol* — протокол пользовательских датаграмм) — один из немногих ключевых элементов набора сетевых протоколов для Интернета. С UDP компьютерные приложения могут посылать сообщения (в данном случае называемые датаграммами) другим хостам по IP-сети без необходимости предварительного сообщения для установки специальных каналов передачи или путей данных. Протокол был разработан Дэвидом П. Ридом в 1980 году и официально определён в RFC 768.

UDP использует простую модель передачи, без явных «рукопожатий» для обеспечения надёжности, упорядочивания или целостности данных. Датаграммы могут прийти не по порядку, дублироваться или вовсе исчезнуть без следа, но гарантируется, что если они придут, то в целостном состоянии. UDP подразумевает, что проверка ошибок и исправление либо не нужны, либо должны исполняться в приложении. Приложения, чувствительные ко времени, но не чувствительные к данным, часто используют UDP, так как предпочтительнее сбросить пакеты, чем ждать задержавшиеся пакеты, что может оказаться невозможным в системах реального времени. При необходимости исправления ошибок на сетевом уровне интерфейса приложение может задействовать TCP или SCTP, разработанные для этой цели.

Природа UDP как протокола без сохранения состояния также полезна для серверов, отвечающих на небольшие запросы от огромного числа клиентов, например DNS и потоковые мультимедийные приложения вроде IPTV, Voice over IP, протоколы туннелирования IP и многие онлайн-игры.

UDP	
	
Название	User Datagram Protocol
Уровень (по модели OSI)	Транспортный
Семейство	TCP/IP (иногда называют UDP/IP)
Создан в	1980 ^[1]
Порт/ID	17 (в IP)
Спецификация	RFC 768 / STD 6 (http://tools.ietf.org/html/std6)
Основные реализации (клиенты)	Ядра Windows, Linux, UNIX
Основные реализации (серверы)	Ядра Windows, Linux, UNIX
Расширяемость	нет
 Медиафайлы на Викискладе	

Содержание

Служебные порты

Структура пакета

Порт отправителя

Порт получателя

Длина датаграммы

Контрольная сумма

Расчёт контрольной суммы

Пример расчёта контрольной суммы

Псевдозаголовки

Псевдозаголовок для IPv4

Псевдозаголовок для IPv6

Надёжность и решения проблемы перегрузок

Приложения

Примеры использования

Сравнение UDP и TCP

См. также

Примечания

Ссылки

Ссылки на RFC

Служебные порты

UDP-приложения используют датаграммные сокеты для установки соединения между хостами. Приложение связывает сокет с его конечной точкой передачи данных, которая является комбинацией IP-адреса и порта службы. Порт — это программная структура, определяемая номером порта — 16-битным целочисленным значением (то есть от 0 до 65 535). Порт 0 зарезервирован, хотя и является допустимым значением порта источника в случае, если процесс-отправитель не ожидает ответных сообщений.

IANA разбила номера портов на три группы.

- Порты с номерами от 0 до 1023 используются для обычных, хорошо известных служб. В Unix-подобных операционных системах для использования таких портов необходимо разрешение суперпользователя.
- Порты с номерами от 1024 до 49 151 предназначены для зарегистрированных IANA служб.
- Порты с 49 152 по 65 535 могут быть использованы для любых целей, поскольку официально не разработаны для какой-то определённой службы. Они также используются как динамические (временные) порты, которые запущенное на хосте программное обеспечение может случайным образом выбрать для самоопределения. По сути, они используются как временные порты в основном клиентами при связи с серверами.

Структура пакета

UDP — минимальный ориентированный на обработку сообщений протокол транспортного уровня, задокументированный в RFC 768.

UDP не предоставляет никаких гарантий доставки сообщения для вышестоящего протокола и не сохраняет состояния отправленных сообщений. По этой причине UDP иногда называют Unreliable Datagram Protocol (англ. — Ненадёжный протокол датаграмм).

UDP обеспечивает многоканальную передачу (с помощью номеров портов) и проверку целостности заголовка и существенных данных (с помощью контрольных сумм). Надёжная передача в случае необходимости должна реализовываться пользовательским приложением.

Биты	0 - 15	16 - 31
0-31	Порт отправителя (Source port)	Порт получателя (Destination port)
32-63	Длина датаграммы (Length)	Контрольная сумма (Checksum)
64-...	Данные (Data)	

Заголовок UDP состоит из четырёх полей, каждое по 2 байта (16 бит). Два из них необязательны к использованию в IPv4 (розовые ячейки в таблице), в то время как в IPv6 необязателен только порт отправителя.

Порт отправителя

В этом поле указывается номер порта отправителя. Предполагается, что это значение задаёт порт, на который при необходимости будет посылаться ответ. В противном же случае значение должно быть равным 0. Если хостом-источником является клиент, то номер порта будет, скорее всего, динамическим. Если источником является сервер, то его порт будет одним из «хорошо известных».

Порт получателя

Это поле обязательно и содержит порт получателя. Аналогично порту отправителя, если хостом-получателем является клиент, то номер порта динамический, если получатель — сервер, то это будет «хорошо известный» порт.

Длина датаграммы

Поле, задающее длину всей датаграммы (заголовок и данных) в байтах. Минимальная длина равна длине заголовка — 8 байт. Теоретически, максимальный размер поля — 65 535 байт для UDP-датаграммы (8 байт на заголовок и 65 527 на данные). Фактический предел для длины данных при использовании IPv4 — 65 507 (помимо 8 байт на UDP-заголовок требуется ещё 20 на IP-заголовок).

На практике также следует учитывать, что если длина IPv4 пакета с UDP будет превышать MTU (для Ethernet по умолчанию 1500 байт), то отправка такого пакета может вызвать его фрагментацию, что может привести к тому, что он вообще не сможет быть доставлен, если промежуточные маршрутизаторы или конечный хост не будут поддерживать фрагментированные IP пакеты. Также в RFC 791 указывается минимальная длина IP пакета 576 байт, которую должны поддерживать все участники IPv4, и рекомендуется отправлять IP пакеты большего размера только в том случае если вы уверены, что принимающая сторона может принять пакеты такого размера. Следовательно, чтобы избежать фрагментации UDP

пакетов (и возможной их потери), размер данных в UDP не должен превышать: $MTU - (Max\ IP\ Header\ Size) - (UDP\ Header\ Size) = 1500 - 60 - 8 = 1432$ байта. Для того чтобы быть уверенным, что пакет будет принят любым хостом, размер данных в UDP не должен превышать: (минимальная длина IP пакета) — $(Max\ IP\ Header\ Size) - (UDP\ Header\ Size) = 576 - 60 - 8 = 508$ байт^[2].

В Jumbogram'мах IPv6 пакеты UDP могут иметь больший размер. Максимальное значение составляет 4 294 967 295 байт ($2^{32} - 1$), из которых 8 байт соответствуют заголовку, а остальные 4 294 967 287 байт — данным.

Следует заметить, что большинство современных сетевых устройств отправляют и принимают пакеты IPv4 длиной до 10000 байт без их разделения на отдельные пакеты. Неофициально такие пакеты называют «Jumbo-пакетами», хотя понятие Jumbo официально относится к IPv6. Тем не менее, «Jumbo-пакеты» поддерживают не все устройства и перед организацией связи с помощью UDP/IP IPv4 посылок с длиной, превышающей 1500 байт, нужно проверять возможность такой связи опытным путём на конкретном оборудовании^[3].

Контрольная сумма

Поле контрольной суммы используется для проверки заголовка и данных на ошибки. Если сумма не сгенерирована передатчиком, то поле заполняется нулями. Поле не является обязательным для IPv4.

Расчёт контрольной суммы

Метод для вычисления контрольной суммы определён в RFC 1071^[4].

Перед расчётом контрольной суммы, если длина UDP-сообщения в байтах нечётна, то UDP-сообщение дополняется в конце нулевым байтом (псевдозаголовок и добавочный нулевой байт не отправляются вместе с сообщением, они используются только при расчёте контрольной суммы). Поле контрольной суммы в UDP-заголовке во время расчёта контрольной суммы принимается нулевым.

Для расчёта контрольной суммы псевдозаголовок и UDP-сообщение разбивается на двухбайтные слова. Затем рассчитывается сумма всех слов в арифметике обратного кода (то есть кода, в котором отрицательное число получается из положительного инверсией всех разрядов числа и существует два нуля: 0x0000 (обозначается +0) и 0xffff (обозначается -0)). Результат записывается в соответствующее поле в UDP-заголовке.

Значение контрольной суммы, равное 0x0000 (+0 в обратном коде), зарезервировано и означает, что для отправки контрольная сумма не вычислялась. В случае, если контрольная сумма вычислялась и получилась равной 0x0000, то в поле контрольной суммы заносят значение 0xffff (-0 в обратном коде).

При получении сообщения получатель считает контрольную сумму заново (уже учитывая поле контрольной суммы), и, если в результате получится -0 (то есть 0xffff), то контрольная сумма считается сошедшейся. Если сумма не сходится (данные были повреждены при передаче, либо контрольная сумма неверно посчитана на передающей

стороне), то решение о дальнейших действиях принимает принимающая сторона. Как правило, в большинстве современных устройств, работающих с UDP/IP-пакетами имеются настройки, позволяющие либо игнорировать такие пакеты, либо пропускать их на дальнейшую обработку, невзирая на неправильность контрольной суммы.

Пример расчёта контрольной суммы

Для примера рассчитаем контрольную сумму нескольких 16-битных слов: 0x398a, 0xf802, 0x14b2, 0xc281.

Для этого можно сначала сложить попарно числа, рассматривая их как 16-разрядные беззнаковые числа с последующим приведением к дополнительному коду путём прибавления единицы к результату, если при сложении произошёл перенос в старший (17-й) разряд (то есть де-факто, этой операцией мы переводим отрицательное число из дополнительного в обратный код). Или, что равноценно, можно считать, что перенос прибавляется к младшему разряду числа.

```
0x398a + 0xf802 = 0x1318c → 0x318d (перенос в старший разряд)
0x318d + 0x14b2 = 0x0463f → 0x463f (число положительное)
0x463f + 0xc281 = 0x108c0 → 0x08c1
```

В конце выполняется инверсия всех битов получившегося числа

$0x08c1 = 0000\ 1000\ 1100\ 0001 \rightarrow 1111\ 0111\ 0011\ 1110 = 0xf73e$ или, иначе — $0xffff - 0x08c1 = 0xf73e$. Это и есть искомая контрольная сумма.

В документе RFC 1071^[4] приведены и другие способы расчёта контрольной суммы, в частности, с использованием 32-разрядной арифметики.

Псевдозаголовки

Псевдозаголовок для IPv4

Если UDP работает над IPv4, контрольная сумма вычисляется при помощи псевдозаголовка, который содержит некоторую информацию из заголовка IPv4. Псевдозаголовок не является настоящим IPv4-заголовком, используемым для отправления IP-пакета. В таблице приведён псевдозаголовок, используемый только для вычисления контрольной суммы.

Биты	0 — 7	8 — 15	16 — 23	24 — 31
0	Адрес источника			
32	Адрес получателя			
64	Нули	Протокол	Длина UDP	
96	Порт источника		Порт получателя	
128	Длина		Контрольная сумма	
160+	Данные			

Адреса источника и получателя берутся из IPv4-заголовка. Значения поля «Протокол» для UDP равно 17 (0x11). Поле «Длина UDP» соответствует длине заголовка и данных.

Вычисление контрольной суммы для IPv4 необязательно, если она не используется, то значение равно 0.

Псевдозаголовок для IPv6

При работе UDP над IPv6 контрольная сумма обязательна. Метод для её вычисления был опубликован в [RFC 2460](#):

При вычислении контрольной суммы опять используется псевдозаголовок, имитирующий реальный IPv6-заголовок:

Биты	0 — 7	8 — 15	16 — 23	24 — 31
0	Адрес источника			
32				
64				
96				
128	Адрес получателя			
160				
192				
224				
256	Длина UDP			
288	Нули			Следующий заголовок
320	Порт источника		Порт получателя	
352	Длина		Контрольная сумма	
384+	Данные			

Адрес источника такой же, как и в IPv6-заголовке. Адрес получателя — финальный получатель; если в IPv6-пакете не содержится заголовка маршрутизации (Routing), то это будет адрес получателя из IPv6-заголовка, в противном случае, на начальном узле, это будет адрес последнего элемента заголовка маршрутизации, а на узле-получателе — адрес получателя из IPv6-заголовка. Значение «Следующий заголовок» равно значению протокола — 17 для UDP. Длина UDP — длина UDP-заголовка и данных.

Надёжность и решения проблемы перегрузок

Из-за недостатка надёжности приложения UDP должны быть готовы к некоторым потерям, ошибкам и дублированиям. Некоторые из них (например, [TFTP](#)) могут при необходимости добавить элементарные механизмы обеспечения надёжности на прикладном уровне.

Но чаще такие механизмы не используются UDP-приложениями и даже мешают им. [Потоковые медиа](#), многопользовательские игры в реальном времени и VoIP — примеры приложений, часто использующих протокол UDP. В этих конкретных приложениях потеря

пакетов обычно не является большой проблемой. Если приложению необходим высокий уровень надёжности, то можно использовать другой протокол (TCP) или воспользоваться методами помехоустойчивого кодирования (Erasure code).

Более серьёзной потенциальной проблемой является то, что в отличие от TCP, основанные на UDP приложения не обязательно имеют хорошие механизмы контроля и избегания перегрузок. Чувствительные к перегрузкам UDP-приложения, которые потребляют значительную часть доступной пропускной способности, могут поставить под угрозу стабильность в Интернете.

Сетевые механизмы были предназначены для того, чтобы свести к минимуму возможные эффекты от перегрузок при неконтролируемых, высокоскоростных нагрузках. Такие сетевые элементы, как маршрутизаторы, использующие пакетные очереди и техники сброса, часто являются единственным доступным инструментом для замедления избыточного UDP-трафика. DCCP (англ. Datagram Congestion Control Protocol — протокол контроля за перегрузками датаграмм) разработан как частичное решение этой потенциальной проблемы с помощью добавления конечному хосту механизмов для отслеживания перегрузок для высокоскоростных UDP-потоков вроде потоковых медиа.

Приложения

Многочисленные ключевые Интернет-приложения используют UDP, в их числе — DNS (где запросы должны быть быстрыми и состоять только из одного запроса, за которым следует один пакет ответа), Простой протокол управления сетями (SNMP), Протокол маршрутной информации (RIP), Протокол динамической конфигурации узла (DHCP).

Голосовой и видеотрафик обычно передаётся с помощью UDP. Протоколы потокового видео в реальном времени и аудио разработаны для обработки случайных потерь пакетов так, что качество лишь незначительно уменьшается вместо больших задержек при повторной передаче потерянных пакетов. Поскольку и TCP, и UDP работают с одной и той же сетью, многие компании замечают, что недавнее увеличение UDP-трафика из-за этих приложений реального времени мешает производительности TCP-приложений вроде систем баз данных или бухгалтерского учёта. Так как и бизнес-приложения, и приложения в реальном времени важны для компаний, развитие качества решений проблемы некоторыми рассматривается в качестве важного приоритета.

Примеры использования

- **Онлайн-игры:** различные гонки и шутеры от первого лица требуют быстрой реакции игрока и как следствие - быстрой передачи данных. UDP обеспечивает низкую задержку в отличие от TCP/IP потому что в нём не требуется упорядочивать передачу данных.
- **Видеотрансляции** (вроде прямых трансляций YouTube или Twitch): данные приложения как правило используют протокол RTP, который в себе содержит пакеты протоколов UDP. В результате достигается передача высокого качества видео, даже при не стабильном интернет соединении.
- **Приложения голосовой связи** (Skype, Zoom, звонки в мессенджерах Telegram и WhatsApp): благодаря UDP достигается низкая задержка и высокое качество звука.

- **Интернет вещей:** датчики температуры и влажности воздуха, а также различные умные лампочки, которые могут менять цвет, зачастую используют протокол UDP. Данные приборы оптимизированы под низкое энергопотребление и обладают маломощным процессором и памятью. Из-за того, что UDP не требует ресурсов на установление соединения, балансировку нагрузки и другие ресурсоемкие операции, удается снизить энергопотребление устройств и предоставить пользователю достаточно полную передачу необходимых данных сберегая вычислительные ресурсы.

Сравнение UDP и TCP

TCP — ориентированный на соединение протокол, что означает необходимость «рукопожатия» для установки соединения между двумя хостами. Как только соединение установлено, пользователи могут отправлять данные в обоих направлениях.

- **Надёжность** — TCP управляет подтверждением, повторной передачей и тайм-аутом сообщений. Производятся многочисленные попытки доставить сообщение. Если оно потеряется на пути, сервер вновь запросит потерянную часть. В TCP нет ни пропавших данных, ни (в случае многочисленных тайм-аутов) разорванных соединений.
- **Упорядоченность** — если два сообщения последовательно отправлены, первое сообщение достигнет приложения-получателя первым. Если участки данных прибывают в неверном порядке, TCP отправляет неупорядоченные данные в буфер до тех пор, пока все данные не могут быть упорядочены и переданы приложению.
- **Тяжеловесность** — TCP необходимо три пакета для установки сокет-соединения перед тем, как отправить данные. TCP следит за надёжностью и перегрузками.
- **Потоковость** — данные читаются как поток байтов, не передается никаких особых обозначений для границ сообщения или сегментов.

UDP — более простой, основанный на сообщениях протокол без установления соединения. Протоколы такого типа не устанавливают выделенного соединения между двумя хостами. Связь достигается путём передачи информации в одном направлении от источника к получателю без проверки готовности или состояния получателя. В приложениях для голосовой связи через интернет-протокол (Voice over IP, TCP/IP) UDP имеет преимущество над TCP, в котором любое «рукопожатие» помешало бы хорошей голосовой связи. В VoIP считается, что конечные пользователи в реальном времени предоставят любое необходимое подтверждение о получении сообщения.

- **Ненадёжный** — когда сообщение посылается, неизвестно, достигнет ли оно своего назначения — оно может потеряться по пути. Нет таких понятий, как подтверждение, повторная передача, тайм-аут.
- **Неупорядоченность** — если два сообщения отправлены одному получателю, то порядок их достижения цели не может быть предугадан.
- **Легковесность** — никакого упорядочивания сообщений, никакого отслеживания соединений и т. д. Это небольшой транспортный уровень, разработанный на IP.
- **Датаграммы** — пакеты посылаются по отдельности и проверяются на целостность только если они прибыли. Пакеты имеют определённые границы, которые соблюдаются после получения, то есть операция чтения на сокете-получателе выдаст сообщение целиком, каким оно было изначально послано.
- **Нет контроля перегрузок** — UDP сам по себе не избегает перегрузок. Приложения, генерирующие большой трафик, могут создать перегрузку сети, если только они не реализуют меры контроля на прикладном уровне.

См. также

- [QUIC](#)
- [UDP Lite](#)
- [SCTP](#)
- [Список портов TCP и UDP](#)

Примечания

1. <https://tools.ietf.org/html/rfc768> (<https://tools.ietf.org/html/rfc768>)
2. *Valentin Plenk*. *Angewandte Netzwerktechnik kompakt: Dateiformate, Übertragungsprotokolle und ihre Nutzung in Java-Applikationen* (<https://books.google.com.ua/books?id=U37gDQAAQB> AJ&pg=PA130). — 1te Aufl. — Springer Vieweg, 2017. — S. 130. — XIV, 194 S. — (IT kompakt). — ISBN 978-3-658-15904-7.
3. *Scott Hogg*. Jumbo Frames. Does your network support Jumbo Frames and should you enable it? // Network World : <http://www.networkworld.com/article/2224722/cisco-subnet/jumbo-frames.html>. — Jun 3, 2013.
4. *R. Braden, D. Borman, C. Partridge*. RFC 1071 — Расчет контрольных сумм в Internet (<http://rfc2.ru/1071.rfc>) (сентябрь 1988). Дата обращения: 3 октября 2014. Архивировано (<https://web.archive.org/web/20141006205923/http://rfc2.ru/1071.rfc>) 6 октября 2014 года.

Ссылки

- Kurose, J. F.; Ross, K. W. (2010). *Computer Networking: A Top-Down Approach* (5th ed.). Boston, MA: Pearson Education. ISBN 978-0-13-136548-3.
- Forouzan, B.A. (2000). *TCP/IP: Protocol Suite*, 1st ed. New Delhi, India: Tata McGraw-Hill Publishing Company Limited.
- content@ipv6.com. «UDP Protocol Overview». [Ipv6.com](http://ipv6.com). Retrieved 17 August 2011.
- Clark, M.P. (2003). *Data Networks IP and the Internet*, 1st ed. West Sussex, England: John Wiley & Sons Ltd.
- [RFC 2675](#)
- Postel, J. (August 1980). [RFC 768: User Datagram Protocol](#). Internet Engineering Task Force. Retrieved from <http://tools.ietf.org/html/rfc768>
- Deering S. & Hinden R. (December 1998). [RFC 2460: Internet Protocol, Version 6 \(IPv6\) Specification](#). Internet Engineering Task Force. Retrieved from <http://tools.ietf.org/html/rfc2460>
- «The impact of UDP on Data Applications». [Networkperformancedaily.com](http://networkperformancedaily.com). Retrieved 17 August 2011.
- [IANA Port Assignments](http://www.iana.org/assignments/port-numbers) (<http://www.iana.org/assignments/port-numbers>)
- *Семенов Ю. А.* [Протокол UDP](http://book.itep.ru/4/44/udp_442.htm) (http://book.itep.ru/4/44/udp_442.htm)
- [Правила чтения таблиц, описывающих структуру заголовков сетевых протоколов](https://wireshark.wiki/wiki/Правила_чтения_таблиц,_описывающих_структуру_заголовков_сетевых_протоколов) (https://wireshark.wiki/wiki/Правила_чтения_таблиц,_описывающих_структуру_заголовков_сетевых_протоколов)

Ссылки на RFC

- [RFC 768](#) — Протокол Пользовательских Датаграмм
- [RFC 2460](#) — Интернет протокол, спецификация версии 6 (IPv6)
- [RFC 2675](#) — IPv6 Jumbograms

- [RFC 4113](#) — Management Information Base for the UDP
 - [RFC 5405](#) — Unicast UDP Usage Guidelines for Application Designers
-

Источник — <https://ru.wikipedia.org/w/index.php?title=UDP&oldid=151182382>

Эта страница в последний раз была отредактирована 15 января 2026 года в 18:01.

Текст доступен по лицензии Creative Commons «С указанием авторства — С сохранением условий» (CC BY-SA); в отдельных случаях могут действовать дополнительные условия.

Wikipedia® — зарегистрированный товарный знак некоммерческой организации «Фонд Викимедиа» (Wikimedia Foundation, Inc.)